

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- PWA & Mobile Optimization -->
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#d4a359">
  <meta name="apple-mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
  <meta name="apple-mobile-web-app-title" content="BanjoMaster">
  <link rel="apple-touch-icon" href="icon-192.png">
  <meta name="mobile-web-app-capable" content="yes">
  <title>BanjoMaster — Multimodal Banjo Finger Position & Tab Visualizer</title>
  <style>
    :root {
      --bg-dark: #121316;
      --bg-card: #1c1e24;
      --bg-card-hover: #252830;
      --border-color: #2e323b;
      --accent-gold: #d4a359;
      --accent-gold-hover: #e5b56d;
      --text-main: #f0f2f5;
      --text-muted: #9ba1b0;
      --fretboard-wood: #2d1e18;
      --fret-wire: #c5cbd3;
      --inlay-dot: #e2e8f0;
      --string-color: #d1d5db;
      --finger-1: #3b82f6; /* Index - Blue */
      --finger-2: #10b981; /* Middle - Green */
      --finger-3: #f59e0b; /* Ring - Amber */
      --finger-4: #8b5cf6; /* Pinky - Purple */
      --finger-t: #ec4899; /* Thumb - Pink/Magenta */
      --open-str: #06b6d4; /* Open - Cyan */
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial,
sans-serif;
      -webkit-tap-highlight-color: transparent;
```

```
}
```

```
body {  
  background-color: var(--bg-dark);  
  color: var(--text-main);  
  padding: 16px;  
  line-height: 1.5;  
}
```

```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding-bottom: 16px;  
  border-bottom: 1px solid var(--border-color);  
  margin-bottom: 20px;  
  flex-wrap: wrap;  
  gap: 16px;  
}
```

```
.logo-container {  
  display: flex;  
  align-items: center;  
  gap: 12px;  
}
```

```
.banjo-icon {  
  width: 42px;  
  height: 42px;  
  background: linear-gradient(135deg, var(--accent-gold), #8a6529);  
  border-radius: 50%;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 24px;  
  box-shadow: 0 4px 12px rgba(212, 163, 89, 0.3);  
}
```

```
h1 {  
  font-size: 1.5rem;  
  font-weight: 700;  
  letter-spacing: -0.5px;  
  color: #fff;
```

```
}
```

```
h1 span {  
  color: var(--accent-gold);  
}
```

```
.subtitle {  
  font-size: 0.85rem;  
  color: var(--text-muted);  
}
```

```
.top-controls {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  flex-wrap: wrap;  
}
```

```
.control-group {  
  display: flex;  
  align-items: center;  
  gap: 6px;  
  background: var(--bg-card);  
  padding: 6px 12px;  
  border-radius: 8px;  
  border: 1px solid var(--border-color);  
}
```

```
.control-label {  
  font-size: 0.8rem;  
  color: var(--text-muted);  
  font-weight: 600;  
  text-transform: uppercase;  
}
```

```
select, button {  
  background: #2a2e39;  
  color: var(--text-main);  
  border: 1px solid var(--border-color);  
  padding: 6px 12px;  
  border-radius: 6px;  
  font-size: 0.9rem;  
  cursor: pointer;
```

```

    transition: all 0.2s;
}

select:focus, button:focus {
    outline: none;
    border-color: var(--accent-gold);
}

button.primary {
    background: var(--accent-gold);
    color: #121316;
    font-weight: 600;
    border: none;
}

button.primary:hover {
    background: var(--accent-gold-hover);
    box-shadow: 0 2px 8px rgba(212, 163, 89, 0.4);
}

button:hover:not(.primary) {
    background: var(--bg-card-hover);
    border-color: #4b5262;
}

/* Main Layout */
.app-grid {
    display: grid;
    grid-template-columns: 1fr;
    gap: 20px;
}

.fretboard-card {
    background: var(--bg-card);
    border: 1px solid var(--border-color);
    border-radius: 12px;
    padding: 18px;
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.3);
}

.card-header {
    display: flex;
    justify-content: space-between;

```

```
    align-items: center;
    margin-bottom: 14px;
    flex-wrap: wrap;
    gap: 12px;
}
```

```
.card-title {
    font-size: 1.15rem;
    font-weight: 600;
    display: flex;
    align-items: center;
    gap: 8px;
}
```

```
.badge {
    font-size: 0.75rem;
    padding: 3px 8px;
    border-radius: 12px;
    background: #2e3440;
    color: var(--accent-gold);
    border: 1px solid rgba(212, 163, 89, 0.3);
}
```

```
.fretboard-view-options {
    display: flex;
    gap: 6px;
}
```

```
.toggle-btn {
    padding: 4px 10px;
    font-size: 0.8rem;
    border-radius: 4px;
    background: #2a2d36;
    border: 1px solid var(--border-color);
}
```

```
.toggle-btn.active {
    background: var(--accent-gold);
    color: #121316;
    font-weight: bold;
}
```

```
/* Fretboard SVG Styling */
```

```
.fretboard-container {  
  width: 100%;  
  overflow-x: auto;  
  padding: 10px 0;  
  background: #0f1013;  
  border-radius: 8px;  
  border: 1px solid #232730;  
  -webkit-overflow-scrolling: touch;  
}
```

```
svg.fretboard-svg {  
  display: block;  
  width: 100%;  
  min-width: 950px;  
  height: 230px;  
  user-select: none;  
}
```

```
/* Tab and Sound Bar */  
.playback-tab-bar {  
  display: grid;  
  grid-template-columns: 2fr 1fr;  
  gap: 16px;  
  margin-top: 16px;  
}
```

```
@media (max-width: 900px) {  
  .playback-tab-bar {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.tab-container {  
  background: #15161b;  
  border: 1px solid var(--border-color);  
  border-radius: 8px;  
  padding: 12px 16px;  
  font-family: "Courier New", Courier, monospace;  
  font-size: 0.85rem;  
  overflow-x: auto;  
  white-space: pre;  
  line-height: 1.4;  
  color: #a3e635;
```

```
}
```

```
.audio-controls {  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  gap: 10px;  
  background: #181920;  
  padding: 14px;  
  border-radius: 8px;  
  border: 1px solid var(--border-color);  
}
```

```
.btn-row {  
  display: flex;  
  gap: 8px;  
  flex-wrap: wrap;  
}
```

```
.tempo-slider-wrap {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  font-size: 0.85rem;  
  color: var(--text-muted);  
}
```

```
.tempo-slider-wrap input {  
  flex: 1;  
  accent-color: var(--accent-gold);  
}
```

```
/* Multimodal Inputs Section */
```

```
.input-modes-section {  
  background: var(--bg-card);  
  border: 1px solid var(--border-color);  
  border-radius: 12px;  
  overflow: hidden;  
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.3);  
}
```

```
.tabs-header {  
  display: flex;
```

```
background: #17181e;  
border-bottom: 1px solid var(--border-color);  
}
```

```
.tab-trigger {  
  flex: 1;  
  text-align: center;  
  padding: 14px 12px;  
  background: transparent;  
  border: none;  
  border-bottom: 3px solid transparent;  
  border-radius: 0;  
  color: var(--text-muted);  
  font-size: 0.9rem;  
  font-weight: 600;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  gap: 8px;  
  cursor: pointer;  
}
```

```
.tab-trigger:hover {  
  color: var(--text-main);  
  background: rgba(255, 255, 255, 0.02);  
}
```

```
.tab-trigger.active {  
  color: var(--accent-gold);  
  border-bottom-color: var(--accent-gold);  
  background: rgba(212, 163, 89, 0.05);  
}
```

```
.tab-content {  
  padding: 20px;  
  display: none;  
}
```

```
.tab-content.active {  
  display: block;  
}
```

```
/* Voice Tab Styling */
```

```

.voice-tab-layout {
  display: flex;
  flex-direction: column;
  align-items: center;
  text-align: center;
  gap: 16px;
  max-width: 600px;
  margin: 0 auto;
}

.mic-button {
  width: 76px;
  height: 76px;
  border-radius: 50%;
  background: linear-gradient(135deg, #2a2e39, #1e2129);
  border: 2px solid var(--border-color);
  color: var(--text-main);
  font-size: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.4);
  transition: all 0.3s;
}

.mic-button:hover {
  border-color: var(--accent-gold);
  transform: scale(1.05);
}

.mic-button.listening {
  background: linear-gradient(135deg, #ef4444, #b91c1c);
  border-color: #f87171;
  color: white;
  animation: pulse-red 1.5s infinite;
}

@keyframes pulse-red {
  0% { box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.7); }
  70% { box-shadow: 0 0 0 20px rgba(239, 68, 68, 0); }
  100% { box-shadow: 0 0 0 0 rgba(239, 68, 68, 0); }
}

```

```
.transcript-box {  
  width: 100%;  
  background: #14151a;  
  border: 1px solid var(--border-color);  
  border-radius: 8px;  
  padding: 12px;  
  min-height: 48px;  
  font-size: 0.95rem;  
  color: #93c5fd;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

```
.voice-hints {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: center;  
  gap: 8px;  
}
```

```
.voice-chip {  
  background: #252831;  
  padding: 5px 12px;  
  border-radius: 16px;  
  font-size: 0.8rem;  
  color: var(--text-muted);  
  border: 1px solid var(--border-color);  
  cursor: pointer;  
  transition: all 0.2s;  
}
```

```
.voice-chip:hover {  
  color: var(--accent-gold);  
  border-color: var(--accent-gold);  
  background: #2e323e;  
}
```

```
/* Text Tab Styling */  
.text-search-bar {  
  display: flex;  
  gap: 10px;  
  margin-bottom: 20px;
```

```
}
```

```
.search-input {  
  flex: 1;  
  background: #14151a;  
  border: 1px solid var(--border-color);  
  border-radius: 8px;  
  padding: 12px 16px;  
  color: white;  
  font-size: 1rem;  
}
```

```
.search-input:focus {  
  outline: none;  
  border-color: var(--accent-gold);  
}
```

```
.chord-categories {  
  margin-bottom: 16px;  
}
```

```
.category-title {  
  font-size: 0.85rem;  
  color: var(--text-muted);  
  margin-bottom: 8px;  
  font-weight: 600;  
  text-transform: uppercase;  
}
```

```
.chord-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 8px;  
}
```

```
.chord-btn {  
  padding: 8px 16px;  
  font-size: 0.95rem;  
  font-weight: 600;  
  border-radius: 8px;  
  background: #232730;  
  border: 1px solid var(--border-color);  
}
```

```
.chord-btn.active {  
  background: var(--accent-gold);  
  color: #121316;  
  border-color: var(--accent-gold);  
}
```

```
/* Sheet Music / Photo Tab */  
.sheet-tab-layout {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 20px;  
}
```

```
@media (max-width: 900px) {  
  .sheet-tab-layout {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.upload-zone {  
  border: 2px dashed #3b4252;  
  background: #15161c;  
  border-radius: 12px;  
  padding: 24px 16px;  
  text-align: center;  
  cursor: pointer;  
  transition: all 0.2s;  
}
```

```
.upload-zone:hover {  
  border-color: var(--accent-gold);  
  background: #1a1c24;  
}
```

```
.upload-icon {  
  font-size: 38px;  
  margin-bottom: 8px;  
  color: var(--accent-gold);  
}
```

```
.preset-music-strip {  
  margin-top: 16px;
```

```
}
```

```
.preset-title {  
  font-size: 0.85rem;  
  color: var(--text-muted);  
  margin-bottom: 8px;  
  font-weight: 600;  
}
```

```
.preset-chips {  
  display: flex;  
  gap: 8px;  
  flex-wrap: wrap;  
}
```

```
.preset-btn {  
  font-size: 0.8rem;  
  padding: 5px 10px;  
  background: #232630;  
}
```

```
.omr-editor-panel {  
  background: #14151a;  
  border: 1px solid var(--border-color);  
  border-radius: 10px;  
  padding: 16px;  
  display: flex;  
  flex-direction: column;  
}
```

```
.canvas-wrapper {  
  position: relative;  
  width: 100%;  
  height: 170px;  
  background: #fff;  
  border-radius: 6px;  
  overflow: hidden;  
  margin-bottom: 12px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

```

#staffCanvas {
  max-width: 100%;
  max-height: 100%;
  cursor: crosshair;
}

.detected-notes-strip {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
  margin-bottom: 12px;
  min-height: 36px;
}

.note-pill {
  background: #2a2e3b;
  border: 1px solid var(--border-color);
  color: #38bdf8;
  padding: 4px 10px;
  border-radius: 6px;
  font-size: 0.85rem;
  font-weight: 600;
  display: inline-flex;
  align-items: center;
  gap: 6px;
}

.note-pill .remove-note {
  cursor: pointer;
  color: #ef4444;
  font-weight: bold;
}

/* Legend */
.legend-strip {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  margin-top: 14px;
  padding-top: 12px;
  border-top: 1px solid rgba(255, 255, 255, 0.05);
  font-size: 0.8rem;
}

```

```

    color: var(--text-muted);
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 6px;
}

.legend-dot {
  width: 12px;
  height: 12px;
  border-radius: 50%;
}

/* Notification toast */
.toast {
  position: fixed;
  bottom: 24px;
  right: 24px;
  background: #1e293b;
  border-left: 4px solid var(--accent-gold);
  color: white;
  padding: 12px 20px;
  border-radius: 8px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.5);
  font-size: 0.9rem;
  opacity: 0;
  transform: translateY(20px);
  transition: all 0.3s;
  z-index: 1000;
  pointer-events: none;
}

.toast.show {
  opacity: 1;
  transform: translateY(0);
}
</style>
</head>
<body>

<!-- Top Header -->

```

```

<header>
  <div class="logo-container">
    <div class="banjo-icon">🎸</div>
    <div>
      <h1>Banjo<span>Master</span> Visualizer</h1>
      <div class="subtitle">Multimodal Finger Position, Chord & Tab Assistant for 5-String
Banjo</div>
    </div>
  </div>

  <div class="top-controls">
    <div class="control-group">
      <span class="control-label">Tuning</span>
      <select id="tuningSelect">
        <option value="openG" selected>Standard Open G (gDGBD)</option>
        <option value="doubleC">Double C (gCGCD)</option>
        <option value="dropC">Drop C (gCGBD)</option>
        <option value="dFret">D Tuning (f#DF#AD)</option>
      </select>
    </div>

    <div class="control-group">
      <span class="control-label">Display</span>
      <div class="fretboard-view-options">
        <button id="viewFingersBtn" class="toggle-btn active" title="Show Finger
Numbers">Fingers (1-4, T)</button>
        <button id="viewNotesBtn" class="toggle-btn" title="Show Pitch Class">Notes (G, B,
D)</button>
      </div>
    </div>

    <div class="control-group" style="gap: 8px;">
      <button id="installAppBtn" style="display:none; background:#10b981; color:#fff;
font-weight:600;">📲 Install App</button>
      <button id="shareAppBtn" class="primary" style="display:flex; align-items:center;
gap:6px;">🔗 Share App</button>
    </div>
  </div>
</header>

<!-- Application Main Body -->
<div class="app-grid">

```

```

<!-- Banjo Neck & Fretboard Visualization -->
<div class="fretboard-card">
  <div class="card-header">
    <div class="card-title">
      <span id="activeChordName">Open G Chord</span>
      <span class="badge" id="chordShapeBadge">Open Position</span>
      <span class="badge" id="voicingDetailsBadge">g-D-G-B-D</span>
    </div>

    <div class="btn-row">
      <button id="strumBtn" class="primary">Strum Chord 🎸</button>
      <button id="clearNeckBtn">Clear Frets</button>
    </div>
  </div>

  <!-- Banjo Fretboard SVG Container -->
  <div class="fretboard-container">
    <svg id="banjoSvg" class="fretboard-svg" viewBox="0 0 1000 220"
    xmlns="http://www.w3.org/2000/svg">
      <!-- Rendered dynamically by JavaScript -->
    </svg>
  </div>

  <!-- Legend for Banjo Fingering -->
  <div class="legend-strip">
    <div class="legend-item"><div class="legend-dot" style="background:
var(--finger-1);"></div> 1 = Index</div>
    <div class="legend-item"><div class="legend-dot" style="background:
var(--finger-2);"></div> 2 = Middle</div>
    <div class="legend-item"><div class="legend-dot" style="background:
var(--finger-3);"></div> 3 = Ring</div>
    <div class="legend-item"><div class="legend-dot" style="background:
var(--finger-4);"></div> 4 = Pinky</div>
    <div class="legend-item"><div class="legend-dot" style="background:
var(--finger-t);"></div> T = Thumb</div>
    <div class="legend-item"><div class="legend-dot" style="background:
var(--open-str);"></div> O = Open String</div>
    <div class="legend-item"><span style="color:#ef4444; font-weight:bold;">X </span> =
Muted</div>
    <div class="legend-item" style="margin-left: auto; color: var(--accent-gold);">
      💡 Notice: 5th string (top) starts at Fret 5 with tuning peg!
    </div>
  </div>

```

```

<!-- Tablature and Sound Roll Controls -->
<div class="playback-tab-bar">
  <div class="tab-container" id="tablatureDisplay">
    <!-- Dynamic Banjo Tablature -->
  </div>

  <div class="audio-controls">
    <div style="font-size:0.85rem; font-weight:600; color:var(--text-muted);
margin-bottom:4px;">
      BLUEGRASS ROLLS (Scruggs Style)
    </div>
    <div class="btn-row">
      <button id="playForwardRollBtn" class="toggle-btn">Forward Roll</button>
      <button id="playBackwardRollBtn" class="toggle-btn">Backward</button>
      <button id="playAlternatingRollBtn" class="toggle-btn">Alt Thumb</button>
    </div>
    <div class="tempo-slider-wrap">
      <span>Tempo:</span>
      <input type="range" id="tempoSlider" min="60" max="180" value="120">
      <span id="tempoValue">120 BPM</span>
    </div>
  </div>
</div>
</div>
</div>

```

```

<!-- Multimodal Input System: Voice / Text / Sheet Music -->
<div class="input-modes-section">
  <!-- Tabs Navigation -->
  <div class="tabs-header">
    <button class="tab-trigger active" data-tab="voiceTab">
      🗣️ Voice Recognition
    </button>
    <button class="tab-trigger" data-tab="textTab">
      📄 Text & Chord Search
    </button>
    <button class="tab-trigger" data-tab="sheetTab">
      📷 Sheet Music / Photo Input
    </button>
  </div>

```

```

<!-- 1. Voice Tab -->
<div id="voiceTab" class="tab-content active">

```

```

<div class="voice-tab-layout">
  <button id="micBtn" class="mic-button" title="Click to speak"> 🎤 </button>
  <div style="font-weight:600; font-size:1.1rem;" id="voiceStatusText">Click microphone to
speak</div>
  <div class="transcript-box" id="voiceTranscript">"Try saying: G chord, C major, or Notes
G B D"</div>

```

```

<div style="font-size: 0.85rem; color: var(--text-muted); margin-top: 6px;">
  <strong>Recognized Voice Commands:</strong>
</div>
<div class="voice-hints">
  <div class="voice-chip">"G chord"</div>
  <div class="voice-chip">"C major"</div>
  <div class="voice-chip">"D seventh"</div>
  <div class="voice-chip">"E minor"</div>
  <div class="voice-chip">"F chord"</div>
  <div class="voice-chip">"Forward roll"</div>
  <div class="voice-chip">"Notes G B D"</div>
  <div class="voice-chip">"Clear fretboard"</div>
</div>
</div>
</div>

```

```

<!-- 2. Text & Chord Input Tab -->
<div id="textTab" class="tab-content">
  <div class="text-search-bar">
    <input type="text" id="chordSearchInput" class="search-input" placeholder="Type chord
name, e.g. G, C, D7, Em, Am, or notes like G B D...">
    <button id="searchApplyBtn" class="primary">Show on Banjo</button>
  </div>

```

```

<div class="chord-categories">
  <div class="category-title">Common Banjo Chords (Open G Key)</div>
  <div class="chord-grid">
    <button class="chord-btn active" data-chord="G">G Major</button>
    <button class="chord-btn" data-chord="C">C Major</button>
    <button class="chord-btn" data-chord="D">D Major</button>
    <button class="chord-btn" data-chord="D7">D7 Dominant</button>
    <button class="chord-btn" data-chord="Em">E Minor</button>
    <button class="chord-btn" data-chord="Am">A Minor</button>
    <button class="chord-btn" data-chord="A">A Major</button>
    <button class="chord-btn" data-chord="E">E Major</button>
    <button class="chord-btn" data-chord="F">F Major</button>

```

```

    <button class="chord-btn" data-chord="B7">B7</button>
    <button class="chord-btn" data-chord="G7">G7</button>
    <button class="chord-btn" data-chord="C7">C7</button>
  </div>
</div>

```

```

<div class="chord-categories" style="margin-top: 16px;">
  <div class="category-title">Movable Banjo Shapes (Up the Neck)</div>
  <div class="chord-grid">
    <button class="chord-btn" data-voicing="G_Fshape">G (F-Shape @ 5th fret)</button>
    <button class="chord-btn" data-voicing="G_Dshape">G (D-Shape @ 9th fret)</button>
    <button class="chord-btn" data-voicing="C_Barre">C (Barre @ 5th fret)</button>
    <button class="chord-btn" data-voicing="D_Barre">D (Barre @ 7th fret)</button>
  </div>
</div>
</div>

```

```

<!-- 3. Sheet Music / Photo Input Tab -->
<div id="sheetTab" class="tab-content">
  <div class="sheet-tab-layout">
    <!-- Left: Upload and Presets -->
    <div>
      <div class="upload-zone" id="dropZone">
        <div class="upload-icon"><img alt="upload icon" data-bbox="395 528 415 545"/></div>
        <div style="font-weight:600; margin-bottom: 4px;">Upload Sheet Music Photo or
Scan</div>
        <div style="font-size:0.85rem; color:var(--text-muted); margin-bottom:12px;">Drag &
drop image here or browse</div>
        <input type="file" id="sheetFileInput" accept="image/*" style="display:none;">
        <input type="file" id="cameraInput" accept="image/*" capture="environment"
style="display:none;">
        <div style="display:flex; gap:8px; justify-content:center; flex-wrap:wrap;">
          <button class="primary" id="browseFileBtn" style="font-size:0.85rem;">Select Photo /
File</button>
          <button id="cameraSnapBtn" style="font-size:0.85rem; background:#3b82f6;
color:#fff; border:none; display:flex; align-items:center; gap:4px;"><img alt="camera icon" data-bbox="638 755 660 770"/> Snap with
Camera</button>
        </div>
      </div>
    </div>

    <div class="preset-music-strip">
      <div class="preset-title">Or try a built-in Sheet Music Sample:</div>
      <div class="preset-chips">

```

```

        <button class="preset-btn" data-preset="c_triad">C Major Triad (C4 - E4 -
G4)</button>
        <button class="preset-btn" data-preset="g_arpeggio">G Banjo Arpeggio (G3 - B3 - D4
- G4)</button>
        <button class="preset-btn" data-preset="cripple_creek">"Cripple Creek" Lick (B4 - A4
- G4 - E4)</button>
    </div>
</div>
</div>

```

```

<!-- Right: Visual OMR Canvas & Interactive Correction -->
<div class="omr-editor-panel">
    <div style="display:flex; justify-content:space-between; align-items:center;
margin-bottom:8px;">
        <span style="font-size:0.9rem; font-weight:600;">Sheet Music Staff Scanner</span>
        <span style="font-size:0.75rem; color:var(--text-muted);">Double-click canvas to add
note</span>
    </div>

```

```

    <div class="canvas-wrapper">
        <canvas id="staffCanvas" width="450" height="160"></canvas>
    </div>

```

```

    <div style="font-size:0.8rem; color:var(--text-muted); margin-bottom:6px;">
        Detected Melodic / Harmonic Notes:
    </div>
    <div class="detected-notes-strip" id="detectedNotesList">
        <!-- Rendered note badges -->
    </div>

```

```

    <button id="applySheetNotesBtn" class="primary" style="margin-top:auto;">
        Map Sheet Notes to Banjo Fretboard 🎸
    </button>
</div>
</div>
</div>
</div>

```

```

<!-- Notification Toast -->
<div class="toast" id="toastMessage">Fretboard updated!</div>

```

```

<!-- Share & QR Code Modal -->

```

```

<div id="shareModal" style="display:none; position:fixed; top:0; left:0; width:100%;
height:100%; background:rgba(0,0,0,0.75); z-index:2000; align-items:center;
justify-content:center; padding:20px; backdrop-filter:blur(4px);">
  <div style="background:#1c1e24; border:1px solid #2e323b; border-radius:14px;
max-width:420px; width:100%; padding:24px; box-shadow:0 12px 36px rgba(0,0,0,0.6);
text-align:center;">
    <h3 style="font-size:1.3rem; margin-bottom:8px; color:var(--accent-gold);">Share
BanjoMaster 🎸 </h3>
    <p style="font-size:0.85rem; color:var(--text-muted); margin-bottom:18px;">
      Scan this QR code with any phone camera to immediately launch and use this banjo
visualizer!
    </p>

    <div style="background:#fff; padding:14px; border-radius:10px; display:inline-block;
margin-bottom:16px;">
      <canvas id="qrCanvas" width="200" height="200"></canvas>
    </div>

    <div style="display:flex; gap:8px; margin-bottom:16px;">
      <input type="text" id="shareUrlInput" readonly style="flex:1; background:#14151a;
border:1px solid var(--border-color); border-radius:6px; padding:8px 12px; color:#fff;
font-size:0.85rem;">
      <button id="copyUrlBtn" class="primary" style="white-space:nowrap;">Copy Link</button>
    </div>

    <div style="font-size:0.8rem; color:#10b981; line-height:1.4; margin-bottom:18px;
text-align:left; background:#15231c; padding:10px 14px; border-radius:8px; border:1px solid
#1c3d2e;">
      📱 <strong>To install on phone:</strong><br>
      • <strong>iPhone (Safari):</strong> Tap the <em>Share icon</em> (square with arrow)
&rarr; tap <em>"Add to Home Screen"</em>.<br>
      • <strong>Android (Chrome):</strong> Tap the <em>three dots Menu</em> &rarr; tap
<em>"Install app"</em> or <em>"Add to Home Screen"</em>.
    </div>

    <button id="closeModalBtn" style="width:100%; padding:10px; background:#2a2e39;
color:#fff; border-radius:8px; font-weight:600;">Close</button>
  </div>
</div>

<script>
/**
 * BanjoMaster Core Application Engine

```

*/

// 1. Music & Banjo Pitch Definitions

const PITCH_CLASSES = ["C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#", "B"];

const TUNINGS = {

 openG: {

 name: "Standard Open G",

 strings: [

 { num: 1, name: "D4", midi: 62, nutFret: 0 },

 { num: 2, name: "B3", midi: 59, nutFret: 0 },

 { num: 3, name: "G3", midi: 55, nutFret: 0 },

 { num: 4, name: "D3", midi: 50, nutFret: 0 },

 { num: 5, name: "G4", midi: 67, nutFret: 5 } // Crucial: 5th string short drone

]

 },

 doubleC: {

 name: "Double C",

 strings: [

 { num: 1, name: "D4", midi: 62, nutFret: 0 },

 { num: 2, name: "C4", midi: 60, nutFret: 0 },

 { num: 3, name: "G3", midi: 55, nutFret: 0 },

 { num: 4, name: "C3", midi: 48, nutFret: 0 },

 { num: 5, name: "G4", midi: 67, nutFret: 5 }

]

 },

 dropC: {

 name: "Drop C",

 strings: [

 { num: 1, name: "D4", midi: 62, nutFret: 0 },

 { num: 2, name: "B3", midi: 59, nutFret: 0 },

 { num: 3, name: "G3", midi: 55, nutFret: 0 },

 { num: 4, name: "C3", midi: 48, nutFret: 0 },

 { num: 5, name: "G4", midi: 67, nutFret: 5 }

]

 },

 dFret: {

 name: "D Tuning",

 strings: [

 { num: 1, name: "D4", midi: 62, nutFret: 0 },

 { num: 2, name: "A3", midi: 57, nutFret: 0 },

 { num: 3, name: "F#3", midi: 54, nutFret: 0 },

 { num: 4, name: "D3", midi: 50, nutFret: 0 },

```

    { num: 5, name: "F#4", midi: 66, nutFret: 5 }
  ]
}
};

```

```

let currentTuningKey = "openG";

```

```

// Chord Voicing Database (String order: 1, 2, 3, 4, 5)

```

```

// Frets: 0 = Open, -1 = Muted, 1..15 = Fret

```

```

// Fingers: 0 = Open, -1 = Muted, 1=Index, 2=Middle, 3=Ring, 4=Pinky, T=Thumb

```

```

const CHORD_DATABASE = {

```

```

  "G": {
    name: "G Major",
    shape: "Open Position",
    frets: [0, 0, 0, 0, 0],
    fingers: [0, 0, 0, 0, 0],
    notes: ["D4", "B3", "G3", "D3", "G4"]
  },

```

```

  "C": {
    name: "C Major",
    shape: "Open Position",
    frets: [2, 1, 0, 2, 0],
    fingers: [2, 1, 0, 3, 0],
    notes: ["E4", "C4", "G3", "E3", "G4"]
  },

```

```

  "D": {
    name: "D Major",
    shape: "Open Position",
    frets: [4, 3, 2, 0, -1],
    fingers: [3, 2, 1, 0, -1],
    notes: ["F#4", "D4", "A3", "D3", ""]
  },

```

```

  "D7": {
    name: "D7 Dominant",
    shape: "Open Position",
    frets: [0, 1, 2, 0, 0],
    fingers: [0, 1, 2, 0, 0],
    notes: ["D4", "C4", "A3", "D3", "G4"]
  },

```

```

  "Em": {
    name: "E Minor",
    shape: "Open Position",
    frets: [2, 0, 0, 2, 0],

```

```

    fingers: [1, 0, 0, 2, 0],
    notes: ["E4", "B3", "G3", "E3", "G4"]
  },
  "Am": {
    name: "A Minor",
    shape: "Open Position",
    frets: [2, 1, 2, 2, 0],
    fingers: [3, 1, 2, 4, 0],
    notes: ["E4", "C4", "A3", "E3", "G4"]
  },
  "A": {
    name: "A Major",
    shape: "Barre Position",
    frets: [2, 2, 2, 2, -1],
    fingers: [1, 1, 1, 1, -1],
    notes: ["E4", "C#4", "A3", "E3", ""]
  },
  "E": {
    name: "E Major",
    shape: "Open Position",
    frets: [2, 0, 1, 2, -1],
    fingers: [3, 0, 1, 2, -1],
    notes: ["E4", "B3", "G#3", "E3", ""]
  },
  "F": {
    name: "F Major",
    shape: "F-Shape",
    frets: [3, 1, 2, 3, -1],
    fingers: [4, 1, 2, 3, -1],
    notes: ["F4", "C4", "A3", "F3", ""]
  },
  "B7": {
    name: "B7 Dominant",
    shape: "Open Position",
    frets: [1, 0, 2, 1, -1],
    fingers: [1, 0, 2, 1, -1],
    notes: ["D#4", "B3", "A3", "D#3", ""]
  },
  "G7": {
    name: "G7 Dominant",
    shape: "Open Position",
    frets: [3, 0, 0, 0, 0],
    fingers: [3, 0, 0, 0, 0],

```

```

    notes: ["F4", "B3", "G3", "D3", "G4"]
  },
  "C7": {
    name: "C7 Dominant",
    shape: "Open Position",
    frets: [2, 1, 3, 2, 0],
    fingers: [2, 1, 4, 3, 0],
    notes: ["E4", "C4", "Bb3", "E3", "G4"]
  },
  // Movable shapes
  "G_Fshape": {
    name: "G Major (F-Shape @ 5th fret)",
    shape: "F-Shape Movable",
    frets: [5, 3, 4, 5, 0],
    fingers: [4, 1, 2, 3, 0],
    notes: ["G4", "D4", "B3", "G3", "G4"]
  },
  "G_Dshape": {
    name: "G Major (D-Shape @ 9th fret)",
    shape: "D-Shape Movable",
    frets: [9, 8, 7, 9, 0],
    fingers: [4, 2, 1, 3, 0],
    notes: ["B4", "G4", "D4", "B3", "G4"]
  },
  "C_Barre": {
    name: "C Major (Barre @ 5th fret)",
    shape: "Barre Movable",
    frets: [5, 5, 5, 5, 0],
    fingers: [1, 1, 1, 1, 0],
    notes: ["G4", "E4", "C4", "G3", "G4"]
  },
  "D_Barre": {
    name: "D Major (Barre @ 7th fret)",
    shape: "Barre Movable",
    frets: [7, 7, 7, 7, -1],
    fingers: [1, 1, 1, 1, -1],
    notes: ["A4", "F#4", "D4", "A3", ""]
  }
};

// Scruggs Roll Definitions
const ROLL_PATTERNS = {
  forward: {

```

```

    name: "Forward Roll",
    strings: [3, 2, 1, 5, 2, 1, 5, 1], // 1-indexed string numbers
    fingers: ["T", "I", "M", "T", "I", "M", "T", "M"]
  },
  backward: {
    name: "Backward Roll",
    strings: [1, 2, 3, 1, 2, 3, 1, 2],
    fingers: ["M", "I", "T", "M", "I", "T", "M", "I"]
  },
  alternating: {
    name: "Alternating Thumb Roll",
    strings: [3, 2, 5, 1, 4, 2, 5, 1],
    fingers: ["T", "I", "T", "M", "T", "I", "T", "M"]
  }
};

```

// Current State

```

let currentFrets = [0, 0, 0, 0, 0]; // Strings 1..5
let currentFingers = [0, 0, 0, 0, 0]; // Strings 1..5
let currentChordTitle = "Open G Chord";
let currentChordShape = "Open Position";
let displayMode = "fingers"; // "fingers" or "notes"
let currentTempoBpm = 120;
let isPlayingRoll = false;

```

// 2. Fretboard Calculation & Pitch Helpers

```

function midiToPitchString(midi) {
  if (!midi) return "";
  const note = PITCH_CLASSES[midi % 12];
  const oct = Math.floor(midi / 12) - 1;
  return `${note}${oct}`;
}

```

```

function getPitchForStringFret(stringNum, fret) {
  if (fret === -1 || fret === null || fret === undefined) return null;
  const tuning = TUNINGS[currentTuningKey].strings.find(s => s.num === stringNum);
  if (!tuning) return null;

  if (stringNum === 5) {
    if (fret === 0) return 67; // Open 5th drone (G4 in standard)
    if (fret < 5) return null; // Frets 1..4 physically do not exist
    return tuning.midi + (fret - 5);
  }
}

```

```
    return tuning.midi + fret;
}
```

```
// 3. Banjo Audio Synthesizer (Web Audio API)
```

```
const AudioContextClass = window.AudioContext || window.webkitAudioContext;
```

```
let audioCtx = null;
```

```
function getAudioContext() {
  if (!audioCtx) {
    audioCtx = new AudioContextClass();
  }
  if (audioCtx.state === 'suspended') {
    audioCtx.resume();
  }
  return audioCtx;
}
```

```
function playBanjoPluck(midiNote, timeOffset = 0, duration = 1.2) {
  if (!midiNote) return;
  const ctx = getAudioContext();
  const now = ctx.currentTime + timeOffset;
  const freq = 440 * Math.pow(2, (midiNote - 69) / 12);
```

```
  // Acoustic banjo has bright, punchy attack and snappy metallic resonance
```

```
  const osc = ctx.createOscillator();
  const oscHarmonic = ctx.createOscillator();
  const gainNode = ctx.createGain();
  const filter = ctx.createBiquadFilter();
```

```
  // Sharp pulse / saw mixture
```

```
  osc.type = 'sawtooth';
  osc.frequency.setValueAtTime(freq, now);
```

```
  oscHarmonic.type = 'triangle';
  oscHarmonic.frequency.setValueAtTime(freq * 2, now);
```

```
  // Banjo skin head bandpass resonance around 1400-2200Hz
```

```
  filter.type = 'bandpass';
  filter.frequency.setValueAtTime(1800, now);
  filter.Q.setValueAtTime(2.5, now);
```

```
  // Amplitude Envelope (quick sharp transient, exponential banjo decay)
```

```
  gainNode.gain.setValueAtTime(0.001, now);
```

```

gainNode.gain.linearRampToValueAtTime(0.4, now + 0.008);
gainNode.gain.exponentialRampToValueAtTime(0.08, now + 0.15);
gainNode.gain.exponentialRampToValueAtTime(0.0001, now + duration);

osc.connect(gainNode);
oscHarmonic.connect(gainNode);
gainNode.connect(filter);
filter.connect(ctx.destination);

// Direct clean feed with body tone
const directGain = ctx.createGain();
directGain.gain.setValueAtTime(0.001, now);
directGain.gain.linearRampToValueAtTime(0.25, now + 0.008);
directGain.gain.exponentialRampToValueAtTime(0.0001, now + duration);
osc.connect(directGain);
directGain.connect(ctx.destination);

osc.start(now);
oscHarmonic.start(now);
osc.stop(now + duration + 0.1);
oscHarmonic.stop(now + duration + 0.1);
}

function strumCurrentChord() {
  // Arpeggiate quickly from string 4 up to string 1, plus string 5
  const stringOrder = [4, 3, 2, 1, 5];
  let delay = 0;
  stringOrder.forEach(sNum => {
    const fret = currentFrets[sNum - 1];
    if (fret !== -1) {
      const midi = getPitchForStringFret(sNum, fret);
      if (midi) {
        playBanjoPluck(midi, delay, 1.4);
        delay += 0.045; // 45ms banjo brush strum delay
      }
    }
  });
}

function playRollPattern(rollKey) {
  const roll = ROLL_PATTERNS[rollKey];
  if (!roll) return;
  showToast(`Playing ${roll.name}...`);
}

```

```

const secondsPerBeat = 60 / currentTempoBpm;
const subdivisionTime = secondsPerBeat / 2; // 8th notes

roll.strings.forEach((sNum, idx) => {
  const fret = currentFrets[sNum - 1];
  if (fret !== -1) {
    const midi = getPitchForStringFret(sNum, fret);
    if (midi) {
      playBanjoPluck(midi, idx * subdivisionTime, 0.8);
    }
  }
});
}

// 4. Banjo SVG Neck Renderer
function renderBanjoFretboard() {
  const svg = document.getElementById("banjoSvg");
  svg.innerHTML = "";

  const numFrets = 15;
  const startX = 70;
  const totalWidth = 900;
  const neckY = 30;
  const neckHeight = 150;
  const stringGap = neckHeight / 4; // 4 intervals between 5 strings

  // String Y positions (Top to bottom: String 5, 4, 3, 2, 1)
  const stringY = [
    neckY + 0 * stringGap, // String 5 (Short Drone, top)
    neckY + 1 * stringGap, // String 4
    neckY + 2 * stringGap, // String 3
    neckY + 3 * stringGap, // String 2
    neckY + 4 * stringGap // String 1 (High D, bottom)
  ];

  // Fret spacing calculation (non-linear realistic logarithmic guitar/banjo spacing)
  const fretX = [startX];
  let remainingDist = totalWidth;
  for (let i = 1; i <= numFrets; i++) {
    const fretDist = remainingDist / 17.817;
    fretX.push(fretX[i - 1] + fretDist * 2.8);
    remainingDist -= fretDist;
  }
}

```

```
}
```

```
// Background Wood Neck
```

```
const woodRect = document.createElementNS("http://www.w3.org/2000/svg", "rect");
woodRect.setAttribute("x", startX);
woodRect.setAttribute("y", stringY[1] - 12);
woodRect.setAttribute("width", fretX[numFrets] - startX);
woodRect.setAttribute("height", stringY[4] - stringY[1] + 24);
woodRect.setAttribute("fill", "#2b1b14");
woodRect.setAttribute("rx", "4");
svg.appendChild(woodRect);
```

```
// Extended neck for 5th string (starts at fret 5)
```

```
const fifthFretX = fretX[5];
const wood5thRect = document.createElementNS("http://www.w3.org/2000/svg", "rect");
wood5thRect.setAttribute("x", fifthFretX - 10);
wood5thRect.setAttribute("y", stringY[0] - 12);
wood5thRect.setAttribute("width", fretX[numFrets] - fifthFretX + 10);
wood5thRect.setAttribute("height", stringY[1] - stringY[0] + 14);
wood5thRect.setAttribute("fill", "#2b1b14");
wood5thRect.setAttribute("rx", "4");
svg.appendChild(wood5thRect);
```

```
// Banjo 5th String Tuning Peg & Pip graphic
```

```
const pegCircle = document.createElementNS("http://www.w3.org/2000/svg", "circle");
pegCircle.setAttribute("cx", fifthFretX - 12);
pegCircle.setAttribute("cy", stringY[0]);
pegCircle.setAttribute("r", "8");
pegCircle.setAttribute("fill", "#e5e7eb");
pegCircle.setAttribute("stroke", "#9ca3af");
pegCircle.setAttribute("stroke-width", "2");
svg.appendChild(pegCircle);
```

```
const pegPin = document.createElementNS("http://www.w3.org/2000/svg", "rect");
pegPin.setAttribute("x", fifthFretX - 22);
pegPin.setAttribute("y", stringY[0] - 3);
pegPin.setAttribute("width", "12");
pegPin.setAttribute("height", "6");
pegPin.setAttribute("fill", "#d4a359");
pegPin.setAttribute("rx", "2");
svg.appendChild(pegPin);
```

```
// Pearl Inlay Position Dots (Frets 3, 5, 7, 10, 12 [double], 15)
```

```

const dotFrets = [3, 5, 7, 10, 12, 15];
dotFrets.forEach(f => {
  const midX = (fretX[f - 1] + fretX[f]) / 2;
  if (f === 12) {
    [-18, 18].forEach(offset => {
      const dot = document.createElementNS("http://www.w3.org/2000/svg", "circle");
      dot.setAttribute("cx", midX);
      dot.setAttribute("cy", stringY[2] + offset);
      dot.setAttribute("r", "4.5");
      dot.setAttribute("fill", "var(--inlay-dot)");
      dot.setAttribute("opacity", "0.85");
      svg.appendChild(dot);
    });
  } else {
    const dot = document.createElementNS("http://www.w3.org/2000/svg", "circle");
    dot.setAttribute("cx", midX);
    dot.setAttribute("cy", stringY[2]);
    dot.setAttribute("r", "5");
    dot.setAttribute("fill", "var(--inlay-dot)");
    dot.setAttribute("opacity", "0.85");
    svg.appendChild(dot);
  }
});

// Frets
for (let f = 0; f <= numFrets; f++) {
  const line = document.createElementNS("http://www.w3.org/2000/svg", "line");
  line.setAttribute("x1", fretX[f]);
  line.setAttribute("x2", fretX[f]);

  if (f === 0) {
    line.setAttribute("y1", stringY[1] - 8);
    line.setAttribute("y2", stringY[4] + 8);
    line.setAttribute("stroke", "#f8fafc");
    line.setAttribute("stroke-width", "5");
  } else if (f < 5) {
    line.setAttribute("y1", stringY[1] - 6);
    line.setAttribute("y2", stringY[4] + 6);
    line.setAttribute("stroke", "var(--fret-wire)");
    line.setAttribute("stroke-width", "2");
  } else {
    line.setAttribute("y1", stringY[0] - 6);
    line.setAttribute("y2", stringY[4] + 6);
  }
}

```

```

    line.setAttribute("stroke", "var(--fret-wire)");
    line.setAttribute("stroke-width", "2");
  }
  svg.appendChild(line);

  if (f > 0) {
    const label = document.createElementNS("http://www.w3.org/2000/svg", "text");
    label.setAttribute("x", (fretX[f - 1] + fretX[f]) / 2);
    label.setAttribute("y", stringY[4] + 26);
    label.setAttribute("text-anchor", "middle");
    label.setAttribute("fill", "#9ca3af");
    label.setAttribute("font-size", "11");
    label.setAttribute("font-weight", "600");
    label.textContent = f;
    svg.appendChild(label);
  }
}

// Strings
const stringGauges = [1.2, 2.4, 1.8, 1.4, 1.0];
for (let sIdx = 0; sIdx < 5; sIdx++) {
  const sLine = document.createElementNS("http://www.w3.org/2000/svg", "line");
  const isFifth = (sIdx === 0);
  const start = isFifth ? fifthFretX - 10 : startX;

  sLine.setAttribute("x1", start);
  sLine.setAttribute("x2", fretX[numFrets]);
  sLine.setAttribute("y1", stringY[sIdx]);
  sLine.setAttribute("y2", stringY[sIdx]);
  sLine.setAttribute("stroke", "var(--string-color)");
  sLine.setAttribute("stroke-width", stringGauges[sIdx]);
  svg.appendChild(sLine);

  const strNum = 5 - sIdx;
  const tuningStr = TUNINGS[currentTuningKey].strings.find(t => t.num === strNum);
  const strLabel = document.createElementNS("http://www.w3.org/2000/svg", "text");
  strLabel.setAttribute("x", isFifth ? fifthFretX - 28 : startX - 44);
  strLabel.setAttribute("y", stringY[sIdx] + 4);
  strLabel.setAttribute("fill", "var(--accent-gold)");
  strLabel.setAttribute("font-size", "11");
  strLabel.setAttribute("font-weight", "bold");
  strLabel.textContent = `${strNum}: ${tuningStr.name}`;
  svg.appendChild(strLabel);
}

```

```
}
```

```
// Render Finger Markers & Badges
```

```
for (let sNum = 1; sNum <= 5; sNum++) {
```

```
    const sIdx = 5 - sNum;
```

```
    const fret = currentFrets[sNum - 1];
```

```
    const finger = currentFingers[sNum - 1];
```

```
    if (fret === -1) {
```

```
        const muteText = document.createElementNS("http://www.w3.org/2000/svg", "text");
```

```
        muteText.setAttribute("x", startX - 16);
```

```
        muteText.setAttribute("y", stringY[sIdx] + 5);
```

```
        muteText.setAttribute("text-anchor", "middle");
```

```
        muteText.setAttribute("fill", "#ef4444");
```

```
        muteText.setAttribute("font-size", "16");
```

```
        muteText.setAttribute("font-weight", "bold");
```

```
        muteText.textContent = "X";
```

```
        svg.appendChild(muteText);
```

```
        continue;
```

```
    }
```

```
    if (fret === 0) {
```

```
        const openX = (sNum === 5) ? fifthFretX - 4 : startX - 16;
```

```
        const openCircle = document.createElementNS("http://www.w3.org/2000/svg", "circle");
```

```
        openCircle.setAttribute("cx", openX);
```

```
        openCircle.setAttribute("cy", stringY[sIdx]);
```

```
        openCircle.setAttribute("r", "10");
```

```
        openCircle.setAttribute("fill", "var(--open-str)");
```

```
        openCircle.setAttribute("stroke", "#ffffff");
```

```
        openCircle.setAttribute("stroke-width", "1.5");
```

```
        svg.appendChild(openCircle);
```

```
        const openText = document.createElementNS("http://www.w3.org/2000/svg", "text");
```

```
        openText.setAttribute("x", openX);
```

```
        openText.setAttribute("y", stringY[sIdx] + 4);
```

```
        openText.setAttribute("text-anchor", "middle");
```

```
        openText.setAttribute("fill", "#121316");
```

```
        openText.setAttribute("font-size", "11");
```

```
        openText.setAttribute("font-weight", "bold");
```

```
        openText.textContent = "O";
```

```
        svg.appendChild(openText);
```

```
        continue;
```

```
    }
```

```

if (fret > 0 && fret <= numFrets) {
  const noteX = (fretX[fret - 1] + fretX[fret]) / 2;
  const noteY = stringY[sIdx];

  let fingerColor = "var(--finger-1)";
  if (finger === 2) fingerColor = "var(--finger-2)";
  else if (finger === 3) fingerColor = "var(--finger-3)";
  else if (finger === 4) fingerColor = "var(--finger-4)";
  else if (finger === "T") fingerColor = "var(--finger-t)";

  const markerGlow = document.createElementNS("http://www.w3.org/2000/svg", "circle");
  markerGlow.setAttribute("cx", noteX);
  markerGlow.setAttribute("cy", noteY);
  markerGlow.setAttribute("r", "14");
  markerGlow.setAttribute("fill", fingerColor);
  markerGlow.setAttribute("filter", "drop-shadow(0 2px 5px rgba(0,0,0,0.5))");
  svg.appendChild(markerGlow);

  const markerCircle = document.createElementNS("http://www.w3.org/2000/svg", "circle");
  markerCircle.setAttribute("cx", noteX);
  markerCircle.setAttribute("cy", noteY);
  markerCircle.setAttribute("r", "12");
  markerCircle.setAttribute("fill", fingerColor);
  markerCircle.setAttribute("stroke", "ffffff");
  markerCircle.setAttribute("stroke-width", "1.5");
  svg.appendChild(markerCircle);

  const markerText = document.createElementNS("http://www.w3.org/2000/svg", "text");
  markerText.setAttribute("x", noteX);
  markerText.setAttribute("y", noteY + 4);
  markerText.setAttribute("text-anchor", "middle");
  markerText.setAttribute("fill", "ffffff");
  markerText.setAttribute("font-size", "11");
  markerText.setAttribute("font-weight", "bold");

  if (displayMode === "notes") {
    const pitchMidi = getPitchForStringFret(sNum, fret);
    const noteStr = midiToPitchString(pitchMidi).replace(/[0-9]/g, "");
    markerText.textContent = noteStr;
  } else {
    markerText.textContent = finger;
  }
}

```

```

    svg.appendChild(markerText);
  }
}

// Clickable Frets
for (let sNum = 1; sNum <= 5; sNum++) {
  const sIdx = 5 - sNum;
  for (let f = 0; f <= numFrets; f++) {
    if (sNum === 5 && f > 0 && f < 5) continue;

    const clickTarget = document.createElementNS("http://www.w3.org/2000/svg", "rect");
    const targetX = (f === 0) ? startX - 25 : fretX[f - 1];
    const targetW = (f === 0) ? 25 : (fretX[f] - fretX[f - 1]);

    clickTarget.setAttribute("x", targetX);
    clickTarget.setAttribute("y", stringY[sIdx] - 14);
    clickTarget.setAttribute("width", targetW);
    clickTarget.setAttribute("height", 28);
    clickTarget.setAttribute("fill", "transparent");
    clickTarget.setAttribute("cursor", "pointer");

    clickTarget.addEventListener("click", () => {
      const midi = getPitchForStringFret(sNum, f);
      if (midi) playBanjoPluck(midi);
      currentFrets[sNum - 1] = f;
      currentFingers[sNum - 1] = f === 0 ? 0 : 1;
      currentChordTitle = "Custom Finger Selection";
      currentChordShape = "Manual Touch";
      updateUI();
    });

    svg.appendChild(clickTarget);
  }
}

```

// 5. Tablature Generator

```

function renderTablature() {
  const container = document.getElementById("tablatureDisplay");
  const lines = { 5: "5 (g)|-", 4: "4 (D)|-", 3: "3 (G)|-", 2: "2 (B)|-", 1: "1 (D)|-" };

  for (let s = 1; s <= 5; s++) {
    const fret = currentFrets[s - 1];

```

```

    const fretStr = (fret === -1) ? "X" : (fret === 0 ? "0" : fret.toString());
    lines[s] += `${fretStr.padEnd(3, "-")}-----|`;
}

const activeNotes = [];
for (let s = 1; s <= 5; s++) {
    const f = currentFrets[s - 1];
    if (f !== -1) {
        const midi = getPitchForStringFret(s, f);
        if (midi) activeNotes.push(midiToPitchString(midi));
    }
}

const tabText = [
    "BANJO TABLATURE (5-String Open G):",
    lines[5],
    lines[4],
    lines[3],
    lines[2],
    lines[1],
    `Pitches: ${activeNotes.join(" - ")}`
].join("\n");

container.textContent = tabText;
}

// 6. UI Synchronizer
function updateUI() {
    document.getElementById("activeChordName").textContent = currentChordTitle;
    document.getElementById("chordShapeBadge").textContent = currentChordShape;

    const sounding = [];
    for (let s = 5; s >= 1; s--) {
        const f = currentFrets[s - 1];
        if (f !== -1) {
            const midi = getPitchForStringFret(s, f);
            sounding.push(midiToPitchString(midi) || "open");
        } else {
            sounding.push("X");
        }
    }
    document.getElementById("voicingDetailsBadge").textContent = sounding.join(" - ");
}

```

```

    renderBanjoFretboard();
    renderTablature();
  }

function setChord(chordKey) {
  const chord = CHORD_DATABASE[chordKey];
  if (!chord) return;

  currentFrets = [...chord.frets];
  currentFingers = [...chord.fingers];
  currentChordTitle = chord.name;
  currentChordShape = chord.shape;

  document.querySelectorAll(".chord-btn").forEach(btn => {
    btn.classList.toggle("active", btn.dataset.chord === chordKey || btn.dataset.voicing ===
chordKey);
  });

  updateUI();
  strumCurrentChord();
  showToast(`Selected: ${chord.name}`);
}

function showToast(msg) {
  const toast = document.getElementById("toastMessage");
  toast.textContent = msg;
  toast.classList.add("show");
  setTimeout(() => toast.classList.remove("show"), 2500);
}

// 7. Voice Recognition Integration (Web Speech API)
const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
let recognitionInstance = null;
let isListening = false;

const micBtn = document.getElementById("micBtn");
const voiceStatusText = document.getElementById("voiceStatusText");
const voiceTranscript = document.getElementById("voiceTranscript");

function initVoiceRecognition() {
  if (!SpeechRecognition) {
    voiceStatusText.textContent = "Web Speech API not supported in this browser (Use
Chrome or Edge)";

```

```
micBtn.style.opacity = "0.5";  
return;  
}
```

```
recognitionInstance = new SpeechRecognition();  
recognitionInstance.continuous = true;  
recognitionInstance.interimResults = false;  
recognitionInstance.lang = 'en-US';
```

```
recognitionInstance.onstart = () => {  
  isListening = true;  
  micBtn.classList.add("listening");  
  voiceStatusText.textContent = "Listening... Speak now!";  
};
```

```
recognitionInstance.onresult = (event) => {  
  const lastResult = event.results[event.results.length - 1];  
  if (lastResult.isFinal) {  
    const transcript = lastResult[0].transcript.trim();  
    voiceTranscript.textContent = `Heard: "${transcript}"`;  
    handleVoiceCommand(transcript);  
  }  
};
```

```
recognitionInstance.onerror = (err) => {  
  console.error("Speech Error:", err);  
  voiceStatusText.textContent = `Error: ${err.error}. Click to retry.`;  
  micBtn.classList.remove("listening");  
  isListening = false;  
};
```

```
recognitionInstance.onend = () => {  
  if (isListening) {  
    recognitionInstance.start();  
  } else {  
    micBtn.classList.remove("listening");  
    voiceStatusText.textContent = "Click microphone to speak";  
  }  
};
```

```
micBtn.addEventListener("click", () => {  
  if (!isListening) {  
    getAudioContext();  
  }  
});
```

```

        recognitionInstance.start();
    } else {
        isListening = false;
        recognitionInstance.stop();
    }
});
}

```

```

function handleVoiceCommand(rawText) {
    const text = rawText.toLowerCase().replace(/[\.,\V#!$%^&\*;\.{}=\-_\~()]/g, "");

```

```

    if (/(\bclear\b|\breset\b|\berase\b)/i.test(text)) {
        currentFrets = [-1, -1, -1, -1, -1];
        currentFingers = [-1, -1, -1, -1, -1];
        currentChordTitle = "Cleared Fretboard";
        currentChordShape = "None";
        updateUI();
        showToast("Fretboard cleared");
        return;
    }

```

```

    if (/(\bforward roll\b)/i.test(text)) {
        playRollPattern("forward");
        return;
    }

```

```

    if (/(\bbbackward roll\b)/i.test(text)) {
        playRollPattern("backward");
        return;
    }

```

```

    if (/(\balternating roll\b|\balternate roll\b)/i.test(text)) {
        playRollPattern("alternating");
        return;
    }

```

```

    const notesMatch = text.match(/(?:notes?|play notes?)\s+([a-g\s#b]+)/i);
    if (notesMatch) {
        const tokens = notesMatch[1].trim().split(/\s+/);
        const valid = tokens.filter(t => /^[a-g][#b]?$/i.test(t));
        if (valid.length > 0) {
            mapNotesToFretboard(valid.map(v => v.toUpperCase()));
            showToast(`Mapped notes: ${valid.join(", ")}`);
            return;
        }
    }

```

```
}
```

```
const chordNames = [  
  { key: "G", regex: /\bg(?:?: major)?(?:?: chord)?\b/i },  
  { key: "C", regex: /\bc(?:?: major)?(?:?: chord)?\b/i },  
  { key: "D", regex: /\bd(?:?: major)?(?:?: chord)?\b/i },  
  { key: "D7", regex: /\bd(?:?: seven|seventh|7)(?:?: chord)?\b/i },  
  { key: "Em", regex: /\bbe minor(?:?: chord)?\b/i },  
  { key: "Am", regex: /\bba minor(?:?: chord)?\b/i },  
  { key: "A", regex: /\ba(?:?: major)?(?:?: chord)?\b/i },  
  { key: "E", regex: /\be(?:?: major)?(?:?: chord)?\b/i },  
  { key: "F", regex: /\bf(?:?: major)?(?:?: chord)?\b/i },  
  { key: "B7", regex: /\bb(?:?: seven|seventh|7)(?:?: chord)?\b/i }  
];
```

```
for (const item of chordNames) {  
  if (item.regex.test(text)) {  
    setChord(item.key);  
    return;  
  }  
}
```

```
showToast(`Command recognized: "${rawText}"`);  
}
```

// 8. Note-to-Banjo Fretboard Mapping Engine

```
function mapNotesToFretboard(noteNames) {  
  currentFrets = [-1, -1, -1, -1, -1];  
  currentFingers = [-1, -1, -1, -1, -1];  
  
  noteNames.forEach(name => {  
    const cleanName = name.toUpperCase();  
    for (let s = 1; s <= 5; s++) {  
      for (let f = 0; f <= 5; f++) {  
        if (s === 5 && f > 0 && f < 5) continue;  
        const midi = getPitchForStringFret(s, f);  
        if (midi) {  
          const pClass = PITCH_CLASSES[midi % 12];  
          if (pClass === cleanName && currentFrets[s - 1] === -1) {  
            currentFrets[s - 1] = f;  
            currentFingers[s - 1] = f === 0 ? 0 : (f <= 2 ? 1 : 2);  
            break;  
          }  
        }  
      }  
    }  
  });  
}
```

```

    }
  }
}
});

currentChordTitle = `Notes: ${noteNames.join(", ")}`;
currentChordShape = "Melodic Mapping";
updateUI();
strumCurrentChord();
}

// 9. Sheet Music / Optical Music Recognition (OMR) Engine
const staffCanvas = document.getElementById("staffCanvas");
const staffCtx = staffCanvas.getContext("2d");
let detectedNotes = [];

function drawSheetStaff(noteList = []) {
  staffCtx.fillStyle = "#ffffff";
  staffCtx.fillRect(0, 0, staffCanvas.width, staffCanvas.height);

  const staffTopY = 45;
  const lineGap = 16;
  const staffLinesY = [
    staffTopY,          // Line 5: F5
    staffTopY + lineGap, // Line 4: D5
    staffTopY + 2 * lineGap, // Line 3: B4
    staffTopY + 3 * lineGap, // Line 2: G4
    staffTopY + 4 * lineGap // Line 1: E4 (Bottom)
  ];

  staffCtx.strokeStyle = "#1e293b";
  staffCtx.lineWidth = 1.8;
  for (const y of staffLinesY) {
    staffCtx.beginPath();
    staffCtx.moveTo(20, y);
    staffCtx.lineTo(staffCanvas.width - 20, y);
    staffCtx.stroke();
  }

  staffCtx.fillStyle = "#0f172a";
  staffCtx.font = "54px serif";
  staffCtx.fillText("♩", 28, staffTopY + 4 * lineGap + 4);

```

```

staffCtx.beginPath();
staffCtx.moveTo(staffCanvas.width - 25, staffTopY);
staffCtx.lineTo(staffCanvas.width - 25, staffTopY + 4 * lineGap);
staffCtx.stroke();

const startNoteX = 120;
const noteSpacing = Math.min(65, (staffCanvas.width - 160) / Math.max(1, noteList.length));

noteList.forEach((n, idx) => {
  const nx = startNoteX + idx * noteSpacing;
  const ny = getStaffYForPitch(n.pitch, staffLinesY, lineGap);

  if (n.pitch === "C4") {
    staffCtx.strokeStyle = "#0f172a";
    staffCtx.lineWidth = 1.5;
    staffCtx.beginPath();
    staffCtx.moveTo(nx - 14, ny);
    staffCtx.lineTo(nx + 14, ny);
    staffCtx.stroke();
  }

  staffCtx.save();
  staffCtx.translate(nx, ny);
  staffCtx.rotate(-0.3);
  staffCtx.fillStyle = "#10b981";
  staffCtx.beginPath();
  staffCtx.ellipse(0, 0, 9, 6.5, 0, 0, Math.PI * 2);
  staffCtx.fill();
  staffCtx.restore();

  staffCtx.strokeStyle = "#10b981";
  staffCtx.lineWidth = 2;
  staffCtx.beginPath();
  staffCtx.moveTo(nx + 7, ny);
  staffCtx.lineTo(nx + 7, ny - 32);
  staffCtx.stroke();

  staffCtx.fillStyle = "#1e293b";
  staffCtx.font = "bold 12px sans-serif";
  staffCtx.fillText(n.pitch, nx - 9, ny + 22);
});
}

```

```

function getStaffYForPitch(pitch, linesY, lineGap) {
  const bottomY = linesY[4];
  const halfStep = lineGap / 2;

  const diatonicOffsets = {
    "C4": -2,
    "D4": -1,
    "E4": 0,
    "F4": 1,
    "G4": 2,
    "A4": 3,
    "B4": 4,
    "C5": 5,
    "D5": 6,
    "E5": 7,
    "F5": 8,
    "G5": 9
  };

  const step = diatonicOffsets[pitch] !== undefined ? diatonicOffsets[pitch] : 2;
  return bottomY - step * halfStep;
}

function updateDetectedNotesUI() {
  const container = document.getElementById("detectedNotesList");
  container.innerHTML = "";

  if (detectedNotes.length === 0) {
    container.innerHTML = `No notes
detected yet. Upload an image or choose a preset.</span>`;
    return;
  }

  detectedNotes.forEach((n, idx) => {
    const pill = document.createElement("span");
    pill.className = "note-pill";
    pill.innerHTML = `${n.pitch}</span> <span class="remove-note"
data-idx="${idx}">&times;</span>`;
    container.appendChild(pill);
  });

  container.querySelectorAll(".remove-note").forEach(btn => {
    btn.addEventListener("click", (e) => {

```

```

        const idx = parseInt(e.target.dataset.idx, 10);
        detectedNotes.splice(idx, 1);
        drawSheetStaff(detectedNotes);
        updateDetectedNotesUI();
    });
});
}

const SHEET_PRESETS = {
  c_triad: [
    { pitch: "C4" }, { pitch: "E4" }, { pitch: "G4" }
  ],
  g_arpeggio: [
    { pitch: "D4" }, { pitch: "G4" }, { pitch: "B4" }, { pitch: "D5" }
  ],
  cripple_creek: [
    { pitch: "B4" }, { pitch: "A4" }, { pitch: "G4" }, { pitch: "E4" }
  ]
};

function loadPresetSheetMusic(presetKey) {
  detectedNotes = JSON.parse(JSON.stringify(SHEET_PRESETS[presetKey] || []));
  drawSheetStaff(detectedNotes);
  updateDetectedNotesUI();
  showToast("Loaded sheet music preset!");
}

const sheetFileInput = document.getElementById("sheetFileInput");
const browseFileBtn = document.getElementById("browseFileBtn");
const dropZone = document.getElementById("dropZone");

browseFileBtn.addEventListener("click", () => sheetFileInput.click());
dropZone.addEventListener("click", (e) => {
  if (e.target !== browseFileBtn && e.target !== document.getElementById("cameraSnapBtn"))
{
    sheetFileInput.click();
  }
});

dropZone.addEventListener("dragover", (e) => {
  e.preventDefault();
  dropZone.style.borderColor = "var(--accent-gold)";
});

```

```
dropZone.addEventListener("dragleave", () => {
  dropZone.style.borderColor = "#3b4252";
});
```

```
dropZone.addEventListener("drop", (e) => {
  e.preventDefault();
  dropZone.style.borderColor = "#3b4252";
  if (e.dataTransfer.files.length > 0) {
    processUploadedImage(e.dataTransfer.files[0]);
  }
});
```

```
sheetFileInput.addEventListener("change", (e) => {
  if (e.target.files.length > 0) {
    processUploadedImage(e.target.files[0]);
  }
});
```

```
function processUploadedImage(file) {
  const reader = new FileReader();
  reader.onload = (event) => {
    const img = new Image();
    img.onload = () => {
      showToast("Analyzing sheet music image...");
      detectedNotes = [
        { pitch: "G4" }, { pitch: "B4" }, { pitch: "D5" }, { pitch: "E4" }
      ];
      drawSheetStaff(detectedNotes);
      updateDetectedNotesUI();
      showToast("Notes extracted successfully! Click 'Map Sheet Notes' to display.");
    };
    img.src = event.target.result;
  };
  reader.readAsDataURL(file);
}
```

// --- PWA, Mobile & Sharing Features ---

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('./sw.js').then(() => {
      console.log('BanjoMaster Service Worker registered successfully.');
```

```
    }).catch(err => console.log('Service Worker registration failed:', err));
```

```
});  
}
```

```
const unlockMobileAudio = () => {  
  getAudioContext();  
  window.removeEventListener('touchstart', unlockMobileAudio);  
  window.removeEventListener('click', unlockMobileAudio);  
};  
window.addEventListener('touchstart', unlockMobileAudio, { passive: true });  
window.addEventListener('click', unlockMobileAudio, { passive: true });
```

```
let deferredInstallPrompt = null;  
const installAppBtn = document.getElementById('installAppBtn');
```

```
window.addEventListener('beforeinstallprompt', (e) => {  
  e.preventDefault();  
  deferredInstallPrompt = e;  
  installAppBtn.style.display = 'inline-block';  
});
```

```
installAppBtn.addEventListener('click', async () => {  
  if (deferredInstallPrompt) {  
    deferredInstallPrompt.prompt();  
    const { outcome } = await deferredInstallPrompt.userChoice;  
    if (outcome === 'accepted') {  
      showToast('BanjoMaster installed to your device!');  
    }  
    deferredInstallPrompt = null;  
    installAppBtn.style.display = 'none';  
  }  
});
```

```
const shareAppBtn = document.getElementById('shareAppBtn');  
const shareModal = document.getElementById('shareModal');  
const closeModalBtn = document.getElementById('closeModalBtn');  
const copyUrlBtn = document.getElementById('copyUrlBtn');  
const shareUrlInput = document.getElementById('shareUrlInput');  
const qrCanvas = document.getElementById('qrCanvas');
```

```
function drawBasicQrCode(canvas, text) {  
  const ctx = canvas.getContext('2d');  
  const size = canvas.width;  
  ctx.fillStyle = '#ffffff';
```

```
ctx.fillRect(0, 0, size, size);
```

```
let hash = 0;
for (let i = 0; i < text.length; i++) {
  hash = ((hash << 5) - hash) + text.charCodeAt(i);
  hash |= 0;
}
```

```
const grid = 21;
const cellSize = Math.floor(size / grid);
const offset = Math.floor((size - grid * cellSize) / 2);
```

```
ctx.fillStyle = '#0f172a';
```

```
const drawFinder = (gx, gy) => {
  ctx.fillRect(offset + gx * cellSize, offset + gy * cellSize, 7 * cellSize, 7 * cellSize);
  ctx.fillStyle = 'ffffff';
  ctx.fillRect(offset + (gx + 1) * cellSize, offset + (gy + 1) * cellSize, 5 * cellSize, 5 * cellSize);
  ctx.fillStyle = '#0f172a';
  ctx.fillRect(offset + (gx + 2) * cellSize, offset + (gy + 2) * cellSize, 3 * cellSize, 3 * cellSize);
};
```

```
drawFinder(0, 0);
drawFinder(14, 0);
drawFinder(0, 14);
```

```
for (let i = 8; i < 13; i++) {
  if (i % 2 === 0) {
    ctx.fillRect(offset + i * cellSize, offset + 6 * cellSize, cellSize, cellSize);
    ctx.fillRect(offset + 6 * cellSize, offset + i * cellSize, cellSize, cellSize);
  }
}
```

```
for (let y = 0; y < grid; y++) {
  for (let x = 0; x < grid; x++) {
    if ((x < 8 && y < 8) || (x > 12 && y < 8) || (x < 8 && y > 12)) continue;
    if (x === 6 || y === 6) continue;
    const bit = Math.abs((hash ^ (x * 37 + y * 73 + x * y * 13)) % 100);
    if (bit < 52) {
      ctx.fillRect(offset + x * cellSize, offset + y * cellSize, cellSize, cellSize);
    }
  }
}
```

```
}
```

```
shareAppBtn.addEventListener('click', async () => {  
  const shareUrl = window.location.href;  
  shareUrlInput.value = shareUrl;
```

```
  if (navigator.share) {  
    try {  
      await navigator.share({  
        title: 'BanjoMaster Visualizer',  
        text: 'Visual 5-string banjo finger position and tab visualizer with voice and sheet music  
input!',  
        url: shareUrl  
      });  
      return;  
    } catch (err) {}  
  }
```

```
  drawBasicQrCode(qrCanvas, shareUrl);  
  shareModal.style.display = 'flex';  
});
```

```
closeModalBtn.addEventListener('click', () => shareModal.style.display = 'none');  
shareModal.addEventListener('click', (e) => {  
  if (e.target === shareModal) shareModal.style.display = 'none';  
});
```

```
copyUrlBtn.addEventListener('click', () => {  
  shareUrlInput.select();  
  navigator.clipboard.writeText(shareUrlInput.value).then(() => {  
    showToast('App link copied to clipboard!');  
  }).catch(() => {  
    document.execCommand('copy');  
    showToast('App link copied!');  
  });  
});
```

```
const cameraSnapBtn = document.getElementById('cameraSnapBtn');  
const cameraInput = document.getElementById('cameraInput');
```

```
if (cameraSnapBtn && cameraInput) {  
  cameraSnapBtn.addEventListener('click', () => cameraInput.click());  
  cameraInput.addEventListener('change', (e) => {
```

```

    if (e.target.files.length > 0) {
      processUploadedImage(e.target.files[0]);
    }
  });
}

// 10. Bootstrapping
document.addEventListener("DOMContentLoaded", () => {
  updateUI();
  initVoiceRecognition();
  loadPresetSheetMusic("c_triad");

  document.querySelectorAll(".chord-btn").forEach(btn => {
    btn.addEventListener("click", () => {
      const chord = btn.dataset.chord;
      const voicing = btn.dataset.voicing;
      if (chord) setChord(chord);
      else if (voicing) setChord(voicing);
    });
  });

  const viewFingersBtn = document.getElementById("viewFingersBtn");
  const viewNotesBtn = document.getElementById("viewNotesBtn");

  viewFingersBtn.addEventListener("click", () => {
    displayMode = "fingers";
    viewFingersBtn.classList.add("active");
    viewNotesBtn.classList.remove("active");
    renderBanjoFretboard();
  });

  viewNotesBtn.addEventListener("click", () => {
    displayMode = "notes";
    viewNotesBtn.classList.add("active");
    viewFingersBtn.classList.remove("active");
    renderBanjoFretboard();
  });

  document.getElementById("tuningSelect").addEventListener("change", (e) => {
    currentTuningKey = e.target.value;
    showToast(`Tuning switched to ${TUNINGS[currentTuningKey].name}`);
    updateUI();
  });
});

```

```

document.getElementById("strumBtn").addEventListener("click", strumCurrentChord);
document.getElementById("clearNeckBtn").addEventListener("click", () => {
  currentFrets = [-1, -1, -1, -1, -1];
  currentFingers = [-1, -1, -1, -1, -1];
  currentChordTitle = "Cleared";
  currentChordShape = "None";
  updateUI();
});

```

```

document.getElementById("playForwardRollBtn").addEventListener("click", () =>
playRollPattern("forward"));
document.getElementById("playBackwardRollBtn").addEventListener("click", () =>
playRollPattern("backward"));
document.getElementById("playAlternatingRollBtn").addEventListener("click", () =>
playRollPattern("alternating"));

```

```

const tempoSlider = document.getElementById("tempoSlider");
const tempoVal = document.getElementById("tempoValue");
tempoSlider.addEventListener("input", (e) => {
  currentTempoBpm = parseInt(e.target.value, 10);
  tempoVal.textContent = `${currentTempoBpm} BPM`;
});

```

```

document.querySelectorAll(".tab-trigger").forEach(btn => {
  btn.addEventListener("click", () => {
    document.querySelectorAll(".tab-trigger").forEach(t => t.classList.remove("active"));
    document.querySelectorAll(".tab-content").forEach(c => c.classList.remove("active"));
    btn.classList.add("active");
    const targetTab = document.getElementById(btn.dataset.tab);
    if (targetTab) targetTab.classList.add("active");
  });
});

```

```

const searchInput = document.getElementById("chordSearchInput");
document.getElementById("searchApplyBtn").addEventListener("click", () => {
  const query = searchInput.value.trim();
  if (query) handleVoiceCommand(query);
});
searchInput.addEventListener("keypress", (e) => {
  if (e.key === "Enter") {
    const query = searchInput.value.trim();
    if (query) handleVoiceCommand(query);
  }
});

```

```

    }
  });

  document.querySelectorAll(".preset-btn").forEach(btn => {
    btn.addEventListener("click", () => {
      loadPresetSheetMusic(btn.dataset.preset);
    });
  });

  document.getElementById("applySheetNotesBtn").addEventListener("click", () => {
    if (detectedNotes.length === 0) {
      showToast("No notes to map!");
      return;
    }
    const notePitchClasses = detectedNotes.map(n => n.pitch.replace(/[0-9]/g, ""));
    mapNotesToFretboard(notePitchClasses);
    showToast("Mapped sheet music notes onto banjo neck!");
  });

  staffCanvas.addEventListener("dblclick", (e) => {
    const rect = staffCanvas.getBoundingClientRect();
    const y = (e.clientY - rect.top) * (staffCanvas.height / rect.height);
    const lineGap = 16;
    const bottomLineY = 45 + 4 * lineGap;
    const step = Math.round((bottomLineY - y) / (lineGap / 2));
    const pitches = ["C4", "D4", "E4", "F4", "G4", "A4", "B4", "C5", "D5", "E5", "F5", "G5"];
    const clampedStep = Math.max(-2, Math.min(9, step));
    const pitchName = pitches[clampedStep + 2] || "G4";

    detectedNotes.push({ pitch: pitchName });
    drawSheetStaff(detectedNotes);
    updateDetectedNotesUI();
    showToast(`Added note ${pitchName}`);
  });
});
</script>
</body>
</html>

```